

# **AI-Powered Appointment Booking with Tool Calling in 4D AIKit**

By Soukaina Bachikh, Customer Success Engineer, 4D Inc.

Technical Note 26-08

## Table of Contents

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Table of Contents .....                                                  | 2  |
| Abstract .....                                                           | 4  |
| Introduction .....                                                       | 4  |
| Tool Calling Concepts .....                                              | 4  |
| 1.1 What is Tool Calling? .....                                          | 4  |
| 1.2 Core Elements.....                                                   | 5  |
| System Prerequisites and Requirements .....                              | 6  |
| 2.1 Prerequisites .....                                                  | 6  |
| 2.2 Requirements.....                                                    | 6  |
| Application Workflow.....                                                | 6  |
| 3.1 High-Level Workflow .....                                            | 6  |
| 3.2 Defining Tools with OpenAITool Class.....                            | 10 |
| 3.3 How the Model Picks a Tool .....                                     | 11 |
| What the Assistant Can Do.....                                           | 11 |
| 4.1 Natural Language Business Queries.....                               | 11 |
| 4.2 Automatic Tool Selection and Dispatch.....                           | 11 |
| 4.3 ORDA-Backed Live Data .....                                          | 11 |
| 4.4 Multi-Tool Chaining.....                                             | 11 |
| 4.5 Secure by Design.....                                                | 12 |
| Implementation .....                                                     | 12 |
| 5.1 Database Structure .....                                             | 12 |
| 5.2 Implementing a Tool: APT_Tool_CheckAvailability.....                 | 13 |
| 5.3 Registering Tools and Sending a Prompt .....                         | 14 |
| 5.4 The System Prompt .....                                              | 14 |
| 5.5 Demo.....                                                            | 14 |
| <b>Beat 1: Live availability</b> .....                                   | 15 |
| <b>Beat 2: Booking a new client</b> .....                                | 16 |
| <b>Beat 3: Confirmation-code lookup</b> .....                            | 17 |
| <b>Beat 4: Cancelling</b> .....                                          | 17 |
| <b>Beat 5: Staff-side: the same business logic, without the AI</b> ..... | 18 |
| <b>Beat 6: Out of scope: what the assistant refuses</b> .....            | 20 |
| Best Practices & Security Considerations.....                            | 20 |
| 6.1 Performance and Scalability .....                                    | 20 |
| 6.2 Security .....                                                       | 21 |

|                                         |    |
|-----------------------------------------|----|
| 6.3 Tool Design Best Practices .....    | 21 |
| 6.4 Troubleshooting Common Issues ..... | 22 |
| Conclusion.....                         | 22 |

## Abstract

This Technical Note demonstrates how tool calling offers an alternative to manual keyword-based intent parsing in a 4D application. It uses a practical demonstration application, demoAPT, to illustrate the pattern in context: a 4D project built around an AI booking assistant for a fictional clinic, “Kestrel Health”. demoAPT exposes its booking operations (checking availability, booking, rescheduling, cancelling, and looking up appointments) as OpenAI tools behind a single public web chat, while a native Admin form gives clinic staff the same operations directly, with no AI involved.

The developer no longer writes logic to detect what the user wants. Instead, the model reads the conversation and, guided by the description supplied with each tool, decides which 4D methods to call and with which arguments. 4D executes the requested methods, which run the underlying ORDA queries, and returns the results as JSON. The model then turns them into a natural-language reply. This Technical Note walks through the full workflow, tool definition with the OpenAITool class, tool design, and the security and cost controls the pattern requires.

## Introduction

This document explains, with a concrete example, how to implement tool calling (also called “function calling”) inside a 4D application. demoAPT illustrates every stage of the pattern: defining tools, dispatching a tool call to 4D code, feeding the result back to the model, and returning the final answer to the end user.

Traditional chatbot logic in business applications relies on the developer anticipating every phrasing a user might type: matching keywords, writing regular expressions, and hand-coding the logic that decides which action a free-text message is asking for. This approach breaks easily: on typos, synonyms, and multi-turn context (“14h30 please” referring to a slot offered two messages earlier), and it does not scale, since every new capability requires new parsing rules.

Tool calling addresses both problems by delegating intent recognition and argument extraction to the language model itself. The developer describes the available actions (“tools”) as JSON schemas, each giving the tool's name, a description of what it does, and the parameters it accepts with their types. The model then decides, from the natural-language conversation, which tool (if any) to call and with which arguments. 4D only needs to declare the tools, execute the requested method when the model asks for one, and return the result as JSON so the model can compose the reply. demoAPT implements this pattern end-to-end for appointment booking, and because that web chat is public and requires no login, it keeps the assistant on topic through the system prompt and limits what a single conversation can spend.

## Tool Calling Concepts

### 1.1 What is Tool Calling?

Given a conversation as input, a large language model (LLM) predicts the most likely text to reply with. It has no access to 4D database. Tool calling (OpenAI's term; also called “function calling”) is what closes that gap: it lets the model ask the application to run a function, then use the answer in its reply.

A request to the Chat Completions API can include a tools array holding one JSON Schema entry per callable function. When the model decides that answering the user requires information or an action outside its own knowledge (e.g. “is Dr. Martin free on Friday?”), it does not guess. Instead it returns a reply whose `finish_reason` is set to `tool_calls`, along with a structured payload naming the tool and its arguments, for example `checkAvailability({date: "2026-08-01", staffID: "..."}).`

**Note:** *finish\_reason* is the status flag present on every Chat Completions response, saying why the model stopped generating.

The application executes that action itself, sends the result back to the model as a role: `tool` message, and calls the model a second time to produce the final natural-language reply, now informed by real data. The model never executes code itself; it only decides what to call and how to phrase the response. In practice this means a single user question can cost two or more full round trips to the AI Provider (one to ask for a tool, one more per tool result) before a reply is ready.

**Note:** The system, user, and assistant message roles are covered in an earlier Tech Tip: <https://kb.4d.com/assetid=79838>. The tool role is specific to tool calling and arrived with the `OpenAITool` class, so it is introduced here.

## 1.2 Core Elements

Any tool-calling implementation needs the following elements. They are roles rather than objects the AIKit component supplies: only the tool definition has a class behind it, `cs.AIKit.OpenAITool`, while the rest are ordinary 4D methods, forms, and web pages that the developer writes.

- **Tool definition:** a JSON Schema describing each callable action (name, description, parameters, required fields), built with `cs.AIKit.OpenAITool.new()`.
- **Dispatcher:** a single-entry point that receives a tool name and arguments from the model and routes to the corresponding business method.
- **Business logic (ORDA):** the 4D methods that query or update the datastore, each returning a plain object. These are ordinary 4D methods: the model can call them, and so can a form, a REST client, or any other part of the application.
- **Conversation loop:** code that holds the message history and calls the model again each time a tool result is added to it.
- **Client interface:** the surface through which the user submits requests, such as a web page served by 4D's web server.
- **Staff interface (optional):** a native 4D form calling the same business methods directly, with no model in the path. Applicable where staff require the same operations without a conversational interface.

Together these elements constitute the complete pattern. Only the set of actions exposed as tools differs between implementations.

## System Prerequisites and Requirements

### 2.1 Prerequisites

- **Working familiarity with the 4D AIKit component:** This note builds on AIKit rather than introducing it. For the fundamentals, refer to the AIKit Tech Note or the 4D AIKit documentation.
- **An OpenAI-compatible AI provider:** Access to an AI service that implements the OpenAI API, together with the corresponding API credentials, is required. 4D AIKit supports a wide range of compatible providers through the same programming interface; only the provider endpoint and authentication settings differ. A current list of supported providers is available in the *Compatible OpenAI Providers* documentation.
- **Provider configuration:** Configure the selected provider by creating the appropriate configuration file (for example, `Resources/AIProvider.json`, or the built-in `AIProviders.json` settings in recent versions of 4D) and supplying the provider endpoint and credentials. The application reads this configuration at runtime, allowing providers to be changed without modifying source code. The configuration file should be excluded from version control and must never contain credentials that are committed, shared in tickets, or included in screenshots.

### 2.2 Requirements

- 4D 21: the minimum version required.
- **The 4D AIKit component (4d/4D-AIKit):** The required dependency, it provides a unified OpenAI-compatible client (`cs.AIKit.OpenAI`) that works with any supported provider through configuration, eliminating the need to construct raw HTTP requests. The component is downloaded automatically by the 4D Component Manager when the project is first opened.
- **4D's built-in web server, enabled:** The assistant is implemented as a web application rather than a native 4D window, so the built-in web server must be running with its document root pointing to the project's WebFolder/.
- **The ORDA data model:** Five tables (Staff, Client, Appointment, Availability, and Conversation) are used throughout the examples. No special schema is required; the tools operate on standard ORDA entities.

## Application Workflow

### 3.1 High-Level Workflow

Every conversation in demoAPT follows the same single path: a browser page, one HTTP request per visitor turn, and a synchronous loop on the 4D side that keeps calling the model until it has a final reply. Figure 1 below is the system-level picture: every box is a real method or table named in this Tech Note. Figure 2 further down replays one concrete conversation through that same picture.

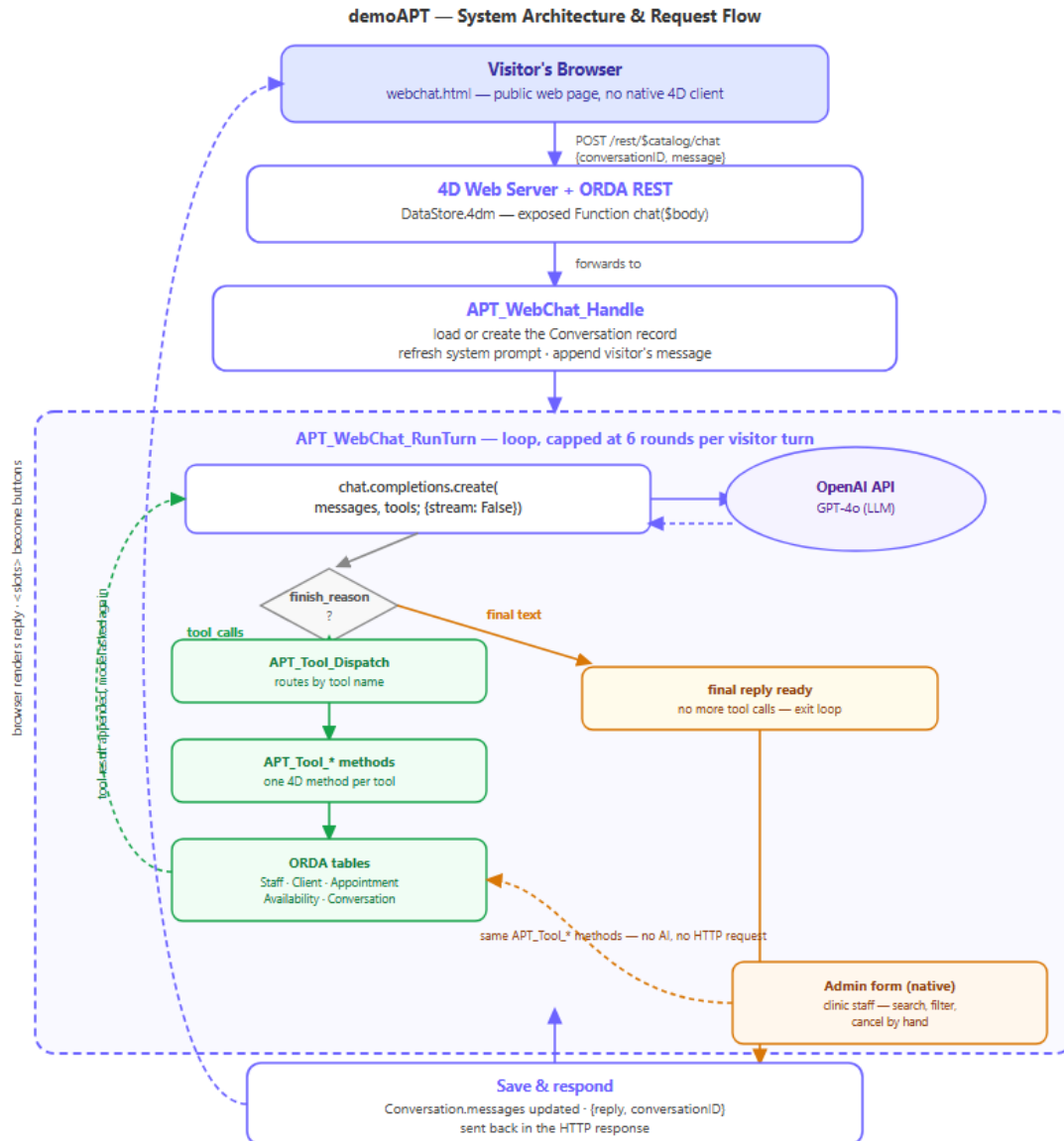


Figure 1 : System architecture (one request, one synchronous loop, two front doors into the same business logic)

The visitor types a message on webchat.html and submits it; the page sends it as a plain HTTP request (a POST of {conversationID, message} via fetch()) to /rest/\$catalog/chat. That address is 4D's ORDA REST mechanism, which routes the request straight to an exposed function called chat in Project/Sources/Classes/DataStore.4dm.

- That function forwards immediately to APT\_WebChat\_Handle, which loads the matching Conversation record by conversationID (or creates a new one on the visitor's first message), refreshes the first entry in the message history with a fresh call to APT\_SystemPrompt (so prompt edits and “today's date” never go stale on a conversation resumed days later), and appends the visitor's message.
- APT\_WebChat\_Handle then calls APT\_WebChat\_RunTurn, which does the actual work: chat.completions.create(...; {stream: False}): a single request and a single response,

with no token-by-token streaming, since a plain HTTP reply has no open connection to stream over anyway.

- If the model's response comes back with `finish_reason = "tool_calls"`, every requested call is routed through `APT_Tool_Dispatch`, and the JSON result is appended to the message history as a role: "tool" message. `APT_WebChat_RunTurn` then calls the model again with that updated history, repeating up to 6 times per visitor turn as a hard safety cap in case the model gets stuck calling tools in circles.
- Once the model finally returns a plain-text reply (no more tool calls), the loop ends. The full updated message history is saved back to the Conversation record, and the reply text travels back to the browser inside the HTTP response, where `webchat.html` renders it, turning any `<slots>` marker into clickable buttons (see Multi-Tool Chaining).
- Off to the side of that whole loop sits the Admin form, a second, entirely independent front door that reaches the same `APT_Tool_*` methods directly, with no model and no HTTP request in between (see Demo Walkthrough).

Figure 2 traces one real conversation through that same architecture (real staff and confirmation-code format, same as the verified demo below), including the `findClient/createClient` branch and the full twelve-tool reference.

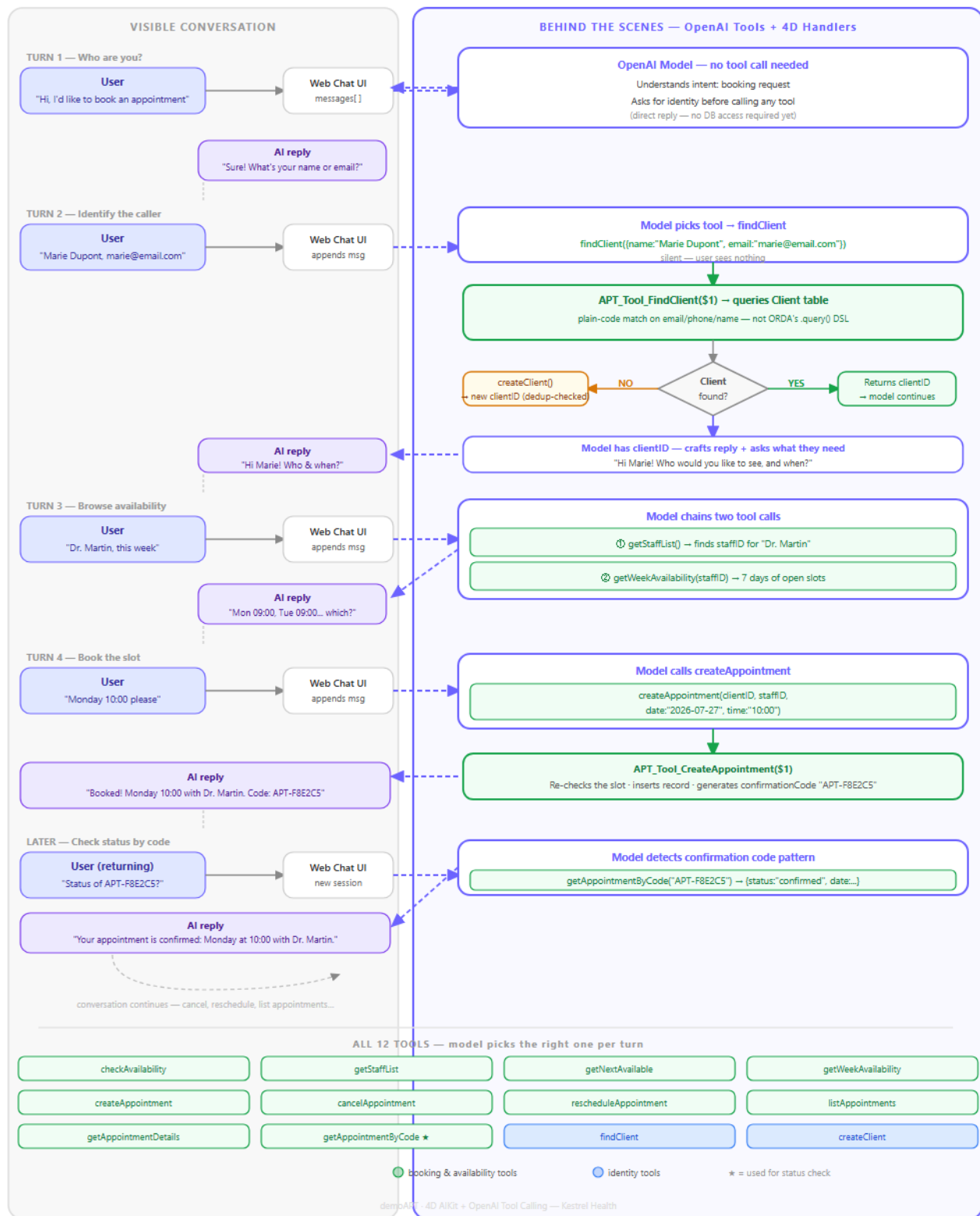


Figure 2: A full conversation (visible chat vs. the tool calls happening behind it)

### 3.2 Defining Tools with OpenAITool Class

All twelve tools are declared in a single method, `APT_GetToolDefinitions`, built with `cs.AIKit.OpenAITool.new()`, the `AIKit` class that wraps one tool's JSON Schema in a 4D object.

The constructor takes a single object with three main keys. `name` is the identifier the model uses when it asks for the tool, and it is what the dispatcher matches on. `description` is prose written for the model, telling it when the tool applies. `parameters` is a JSON Schema object: a type, a properties map of typed arguments each with its own description, and a required collection naming which of those arguments are mandatory (Implementing a Tool: `APT_Tool_CheckAvailability` shows all three filled in).

Each call returns one `OpenAITool` instance, `APT_GetToolDefinitions` collects the twelve of them into a collection, and that collection is what gets passed as the “tools” parameter of `chat.completions.create()` (see [Registering Tools and Sending a Prompt](#) section).

The constructor also accepts a handler key, which is how `AIKit`'s own tool-calling helper, `registerTools()`, expects tools to be supplied: the handler holds the code `AIKit` invokes on the project's behalf when the model asks for that tool. `demoAPT` leaves handler off and does not call `registerTools()`, because dispatch is done manually through `APT_Tool_Dispatch` (see [Automatic Tool Selection and Dispatch](#)), since the same business methods must stay directly callable from the Admin form with no model in the path. A project with only one front door into its business logic will write less code with the handler route.

Each definition is deliberately explicit about what it does (the description is the model's signal for when to use the tool), which parameters are required versus optional and their expected format (e.g. dates as YYYY-MM-DD), and sequencing hints written directly into the description, for example `createAppointment`'s “findClient must be called first”, and `getWeekAvailability`'s “Use this to proactively suggest alternative days when a specifically requested date/time is not available, instead of just reporting failure.”

In table below, more details about the purpose of each tool :

| Tool                               | Purpose                                                                                                      |
|------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>checkAvailability</code>     | Free slots for a date (optional staffID filter)                                                              |
| <code>getStaffList</code>          | Active staff (optional specialty filter)                                                                     |
| <code>getNextAvailable</code>      | Next free slot for one staff member from a date                                                              |
| <code>getWeekAvailability</code>   | Free slots across the next 7 days for one staff member; powers proactive alternatives                        |
| <code>createAppointment</code>     | Books a slot, generates an APT-XXXXXX code                                                                   |
| <code>cancelAppointment</code>     | Sets status to cancelled                                                                                     |
| <code>rescheduleAppointment</code> | Moves an appointment to a new date/time after a slot check                                                   |
| <code>listAppointments</code>      | Client's appointments (upcoming / past / all)                                                                |
| <code>getAppointmentDetails</code> | Full detail by internal appointment ID                                                                       |
| <code>getAppointmentByCode</code>  | Lookup by APT-XXXXXX confirmation code                                                                       |
| <code>findClient</code>            | Search by name/email/phone; always called before <code>createAppointment</code> or <code>createClient</code> |

| Tool         | Purpose                                                      |
|--------------|--------------------------------------------------------------|
| createClient | Register a new client; only when findClient returns no match |

### 3.3 How the Model Picks a Tool

For a tool set of this size, the model's native reasoning over each tool's name, description, and parameter schema against the conversation is sufficient, and it is handled entirely inside the OpenAI API. The model's tool choice is effectively a semantic match between the user's intent and the tool descriptions supplied in the request.

**Note:** A Tech Tip covers *tool\_choice*, for cases where forcing a specific tool is required:  
<https://kb.4d.com/assetid=80052>

## What the Assistant Can Do

### 4.1 Natural Language Business Queries

Clients interact entirely in plain English, as in “Do you have anything with Dr. Bernard next Tuesday afternoon?”, with no menus, forms, or fixed commands. The system prompt returned by APT\_SystemPrompt establishes tone, injects the actual current date every turn (the model has no other way to know “today” and will otherwise guess), and handles partial information and follow-up context from one turn to the next.

### 4.2 Automatic Tool Selection and Dispatch

The developer never writes an If (text contains "cancel") branch. APT\_Tool\_Dispatch is a single, flat Case of that maps a tool name to a 4D method; the decision of which branch to take was already made by the model. The Admin form's own Cancel button calls APT\_Tool\_CancelAppointment directly, the identical method the AI calls, so staff and AI can never disagree about what cancelling an appointment actually does.

### 4.3 ORDA-Backed Live Data

Every tool call reads or writes through ORDA (ds.Staff, ds.Client, ds.Appointment, ds.Availability, ds.Conversation); the assistant never sees stale or hard-coded data.

For instance, checkAvailability and getWeekAvailability both compute free slots live through the shared APT\_ComputeFreeSlots helper, cross-referencing a staff member's weekly Availability against already-booked Appointment records.

### 4.4 Multi-Tool Chaining

Several business rules require more than one tool call per user request, and the model chains them without being told to explicitly at runtime, because the sequencing is embedded in APT\_SystemPrompt and the tool descriptions: findClient before createAppointment; createClient only if findClient finds nothing; checkAvailability or getNextAvailable before offering or confirming a slot; and, when a requested date/time is unavailable, an immediate getWeekAvailability call to offer real alternatives instead of a dead end. When the model

presents concrete bookable options this way, it appends a machine-readable <slots> block after its sentence. webchat.html strips this from the visible text and renders it as clickable pill buttons, so picking one sends a plain-language follow-up (“Book me for Tue, Jul 21 at 14:30 with Dr. X.”) right back through the same chain.

## 4.5 Secure by Design

- Internal UUIDs (clientID, staffID, appointmentID) are never surfaced to the client; only the human-readable confirmationCode (APT-XXXXXX) is ever shown or spoken by the assistant.
- The OpenAI API key lives in Resources/AIProvider.json (gitignored, read by APT\_GetOpenAIKey), never hard-coded and never exposed to the public webchat.html.
- A system-prompt scope guard declines and redirects any request unrelated to appointment booking, with no tool calls at all, added specifically to stop the public, unauthenticated web chat from being used as a free general-purpose chatbot and running up AI Provider cost.
- Hard cost caps back up the prompt: every chat.completions.create() call passes max\_tokens: 500, and the tool-calling loop is capped at 6 rounds per visitor turn.
- createClient re-checks email/phone against APT\_FindClientEntity before registering a new client, so a model that skips the findClient step still can't create a duplicate; it quietly resolves to the existing client instead.

## Implementation

### 5.1 Database Structure

UUIDs are never exposed to end users; only the human-readable confirmationCode is shown.

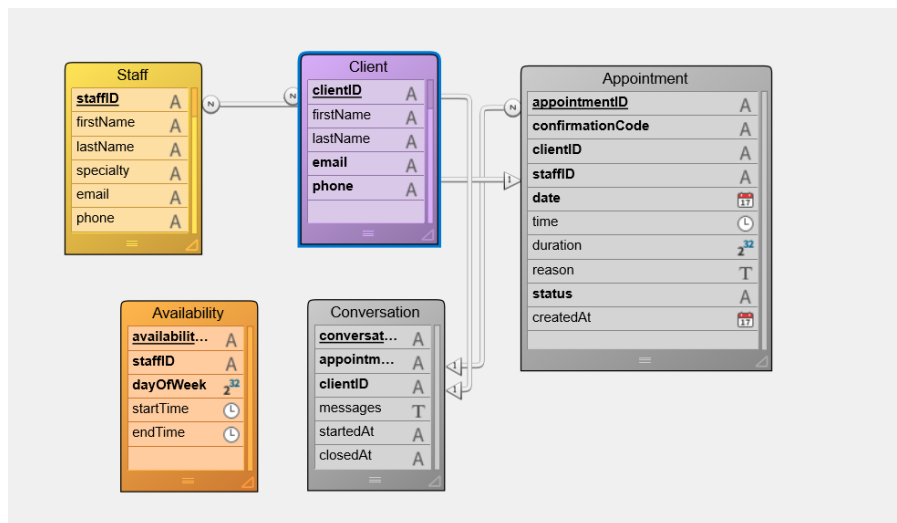


Figure 2 : Database structure

| Table | Primary Key | Key Fields                                             |
|-------|-------------|--------------------------------------------------------|
| Staff | staffID     | firstName, lastName, specialty, slotDuration, isActive |

| Table        | Primary Key    | Key Fields                                                                             |
|--------------|----------------|----------------------------------------------------------------------------------------|
| Client       | clientID       | firstName, lastName, email, phone, createdAt                                           |
| Appointment  | appointmentID  | confirmationCode (APT-XXXXXX), clientID, staffID, date, time, duration, reason, status |
| Availability | availabilityID | staffID, dayOfWeek (1=Mon...7=Sun, ISO), startTime, endTime                            |
| Conversation | conversationID | appointmentID, clientID, messages (JSON text), startedAt, closedAt (text timestamps)   |

## 5.2 Implementing a Tool: APT\_Tool\_CheckAvailability

As a concrete example, checkAvailability is declared in APT\_GetToolDefinitions using AIKit's simplified tool constructor:

```
$tools.push(cs.AIKit.OpenAITool.new({\
  name: "checkAvailability";\
  description: "Lists free appointment slots for a given date, optionally
    filtered to one staff member.";\
  parameters: {\
    type: "object";\
    properties: {\
      date: {type: "string"; description: "Date to check, formatted YYYY-
MM-DD"};\
      staffID: {type: "string"; description: "Optional staff member UUID
to filter to"}\
    };\
    required: ["date"]\
  }\
}))
```

When the model calls this tool, APT\_Tool\_Dispatch routes to the handler, which loops the matching (active) staff members and delegates the actual slot math to the shared APT\_ComputeFreeSlots helper:

```
#DECLARE($params : Object) : Object

var $result : Object := {date: $params.date; slots: []}

var $staffList : cs.StaffSelection
If ($params.staffID # Null) && ($params.staffID # "")
  $staffList := ds.Staff.query("staffID = :1 and isActive = :2";
$params.staffID; True)
Else
  $staffList := ds.Staff.query("isActive = :1"; True)
End if

var $staff : cs.StaffEntity
For each ($staff; $staffList)
  var $freeSlots : Collection
  $freeSlots := APT_ComputeFreeSlots($staff; Date($params.date))
  // ... push {staffID, staffName, time} for each free slot
End for each
```

```
return $result
```

The handler returns a plain object, never a formatted sentence, so the model is free to phrase the reply however best fits the conversation, and the same APT\_ComputeFreeSlots helper is reused by getWeekAvailability to search across seven days instead of one.

### 5.3 Registering Tools and Sending a Prompt

APT\_WebChat\_Handle refreshes the system message every turn, rather than trusting whatever was stored when the conversation began, and appends the visitor's message to the history:

```
If ($messages.length>0) && ($messages[0].role="system")
  $messages[0].content := APT_SystemPrompt
Else
  $messages.unshift({role: "system"; content: APT_SystemPrompt})
End if
$messages.push({role: "user"; content: $userMessage})
```

APT\_WebChat\_RunTurn then registers the tool list on the actual call, by passing APT\_GetToolDefinitions directly into the tools parameter, alongside a hard max\_tokens cap:

```
var $openAI : cs.AIKit.OpenAI
$openAI := cs.AIKit.OpenAI.new(APT_GetOpenAIKey)

var $completion : cs.AIKit.OpenAIChatCompletionsResult
$completion := $openAI.chat.completions.create($messages; {\
  model: "gpt-5";\
  tools: APT_GetToolDefinitions;\
  stream: False;\
  max_tokens: 500\
})
```

**Note:** This call sits inside a While loop (see High-Level Workflow) that re-runs it after each dispatched tool call, up to 6 times, until \$completion.choice.finish\_reason is no longer "tool\_calls". stream: False means the whole call blocks until the complete reply is ready.

### 5.4 The System Prompt

APT\_SystemPrompt returns the single string that is re-injected as message[0] on every turn. It carries the whole behavioural contract: scope, date grounding, the tool-ordering rules the model must respect, the ban on exposing UUIDs, and the machine-readable <slots> block the web page parses into clickable buttons. None of this is enforced by the UI; the dispatcher runs whatever the model asks for, so this text is where the guardrails live.

### 5.5 Demo

This demo was run live against a freshly seeded database before writing it down. It doubles as a script for presenting demoAPT to an audience: what to type, which tool fires, what 4D does

with it, and what to point at while it happens. The public Kestrel Health site exposes a live “Tool Activity” panel that renders this trace on screen once a reply lands (all at once, since the endpoint is synchronous, this panel has been added for demo purposes, to understand exactly which tool(s) has/have been called and when).

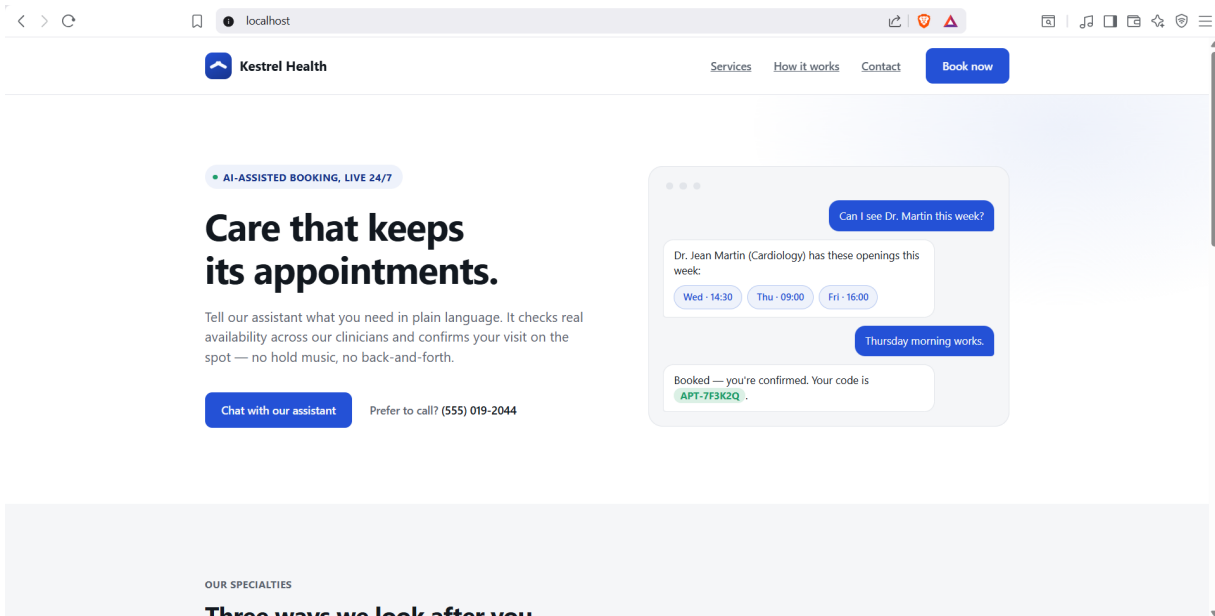


Figure 3: The Kestrel Health landing page, with the assistant's own example conversation

**Before starting:** run APT\_Seed to reset to a clean, predictable dataset (this script's tool arguments assume a fresh seed; dates are relative to whenever it runs). Then open the assistant at <http://localhost/webchat.html> with 4D's web server running, and optionally open the native Admin form (APT\_Open("admin")) side by side to show the clinic's own view of the same data. Point out the Tool Activity panel once, before the first message.

| Seeded Staff       | Specialty · Slot          |
|--------------------|---------------------------|
| Dr. Jean Martin    | Cardiology · 30 min       |
| Dr. Claire Bernard | General Medicine · 20 min |
| Paul Dupuis        | Support · 15 min          |

| Seeded Clients    | Existing Appointment |
|-------------------|----------------------|
| Marie Dupont      | APT-DEMO01           |
| Thomas Leroy      | APT-DEMO02           |
| Nathalie Rousseau | APT-DEMO03           |
| Antoine Moreau    | APT-DEMO04           |

## Beat 1: Live availability

Type / say:

“Can I see Dr. Martin this week?”

**Fires:** getStaffList, then getWeekAvailability

```

getStaffList → {}
↳ 3 active staff returned (Martin, Bernard, Dupuis)
getWeekAvailability → {staffID: "<Jean Martin's UUID>"}
↳ 7 days of open slots returned for Jean Martin

```

**Narration cue:** “Notice the model didn't just guess a schedule: it first asked 4D who our staff actually are, then pulled Dr. Martin's real week from the Availability and Appointment tables.”

Why it lands: two tool calls, back to back, with no extra prompting. The model resolved “Dr. Martin” to a real staffID on its own before asking for his schedule.

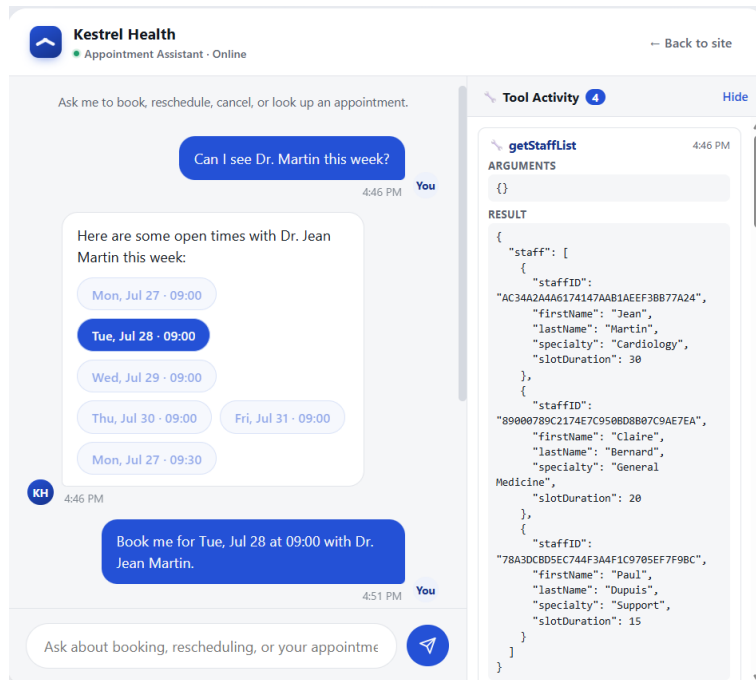


Figure 4: Tool Activity panel showing the `getStaffList` call and result

## Beat 2: Booking a new client

Type / say:

“My name is Sophie Laurent, email `sophie.laurent@example.com`”

**Fires:** `findClient`, `createAppointment`

```

findClient → {name: "Sophie Laurent", email: "..."}
↳ no match
createClient → {firstName: "Sophie", lastName: "Laurent", ...}
↳ new clientID issued
createAppointment → {clientID, staffID, date, time: "10:00"}
↳ confirmed, code APT-F8E2C5

```

**Narration cue:** “Watch the order: it checks for an existing client before creating one, and only books once it has a real clientID. That's a rule in the system prompt, not something the UI enforces.”

Why it lands: three chained calls in one turn, in the exact sequence the system prompt requires: a concrete example of the model reasoning through a multi-step business rule unassisted.

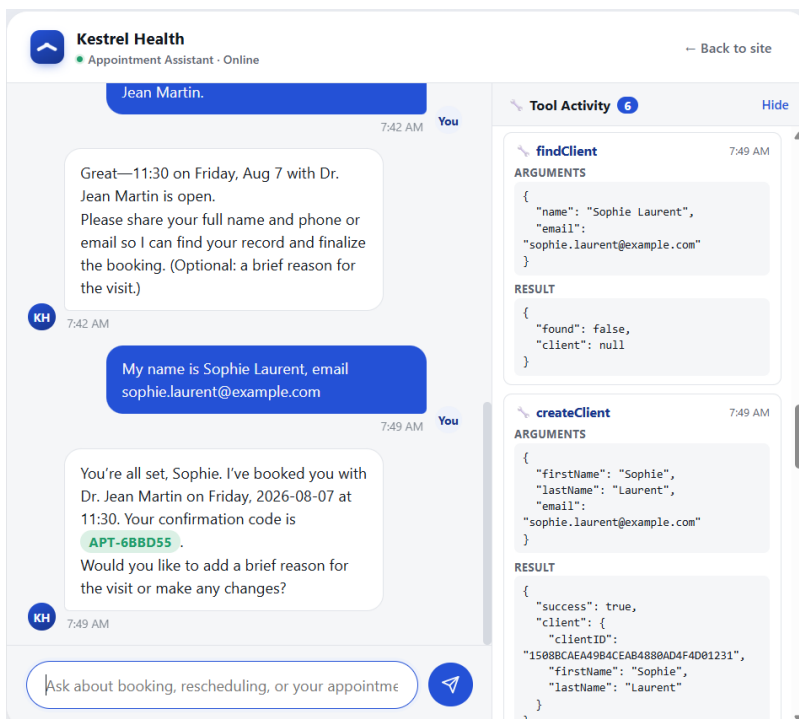


Figure 5: findClient and createClient in the Tool Activity panel, with the confirmation reply and code pill

### Beat 3: Confirmation-code lookup

Type / say:

“Can you look up my appointment APT-F8E2C5?”

Fires: getAppointmentByCode

```
getAppointmentByCode → {confirmationCode: "APT-F8E2C5"}  
↳ full appointment record returned, no UUID shown to the user
```

**Narration cue:** “One call, straight to the point, and the reply never mentions an internal ID, only the code.”

Why it lands: a clean single-tool turn right after a three-tool one is a good contrast beat: the model calls exactly as many tools as the question needs, no more.

### Beat 4: Cancelling

Type / say:

“Actually please cancel that appointment.”

Fires: cancelAppointment

```
cancelAppointment → {appointmentID: "..."}  
↳ status set to cancelled
```

**Narration cue:** “‘That appointment’: the model carried the right appointment forward from two turns ago without being told the code or ID again.”

Why it lands: closes the loop on the exact appointment booked in Beat 2, demonstrating multi-turn memory without any explicit ID in the user's message.

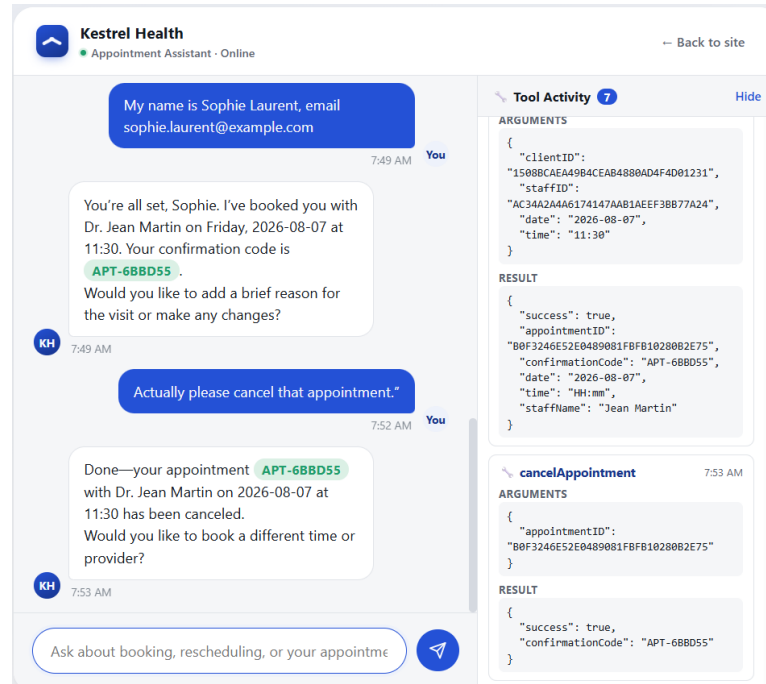


Figure 6: `cancelAppointment` in the Tool Activity panel, with the cancellation reply

### Beat 5: Staff-side: the same business logic, without the AI

Switch to the native Admin form (`APT_Open("admin")`) to show the same data from the clinic's side: a search field, filter buttons (Upcoming / Past / All / Cancelled), a listbox of appointments bound to `Form.appointments`, and a detail panel for the selected row.

- Filter to Cancelled: Sophie Laurent's appointment from Beat 4 is already sitting there, cancelled a few minutes ago by the AI. Same table, same status field, no separate “AI cancellations” log to reconcile.

**Appointment Manager**  
Search, filter, and review appointments

Search by code or client name...

Buttons: Search, All, Upcoming, Past, Cancelled

| Code       | Date       | Time  | Client            | Staff          | Status    |
|------------|------------|-------|-------------------|----------------|-----------|
| APT-68BD55 | 2026-08-07 | 11:30 | Sophie Laurent    | Jean Martin    | cancelled |
| APT-DEMO08 | 2026-07-16 | 09:30 | Nathalie Rousseau | Claire Bernard | cancelled |
| APT-14814C | 2026-07-14 | 12:30 | John Antoine      | Jean Martin    | cancelled |
| APT-DEMO07 | 2026-07-12 | 13:00 | Antoine Moreau    | Paul Dupuis    | cancelled |

**Appointment detail**

Code: APT-68BD55

Date / Time: 2026-08-07 11:30

Client: Sophie Laurent

Staff: Jean Martin

Status: **cancelled**

Reason:

Cancel appointment

Figure 7: The Admin form (filters, listbox, and detail panel with Cancel appointment)

- Now filter to Upcoming and select Thomas Leroy's seeded appointment (APT-DEMO02) instead (a booking the AI never touched) and click Cancel. AdminForm\_CancelSelected calls APT\_Tool\_CancelAppointment({appointmentID: ...}) directly, the exact same method the AI called on Sophie's appointment in Beat 4, then reloads the list.

**Appointment Manager**  
Search, filter, and review appointments

Search by code or client name...

Buttons: Search, All, Upcoming, Past, Cancelled

| Code       | Date       | Time  | Client       | Staff          | Status    |
|------------|------------|-------|--------------|----------------|-----------|
| APT-DEMO02 | 2026-08-06 | 09:00 | Thomas Leroy | Claire Bernard | confirmed |

**Appointment detail**

Code: APT-DEMO02

Date / Time: 2026-08-06 09:00

Client: Thomas Leroy

Staff: Claire Bernard

Status: **confirmed**

Reason: Annual physical

Cancel appointment

Figure 8: The Admin form (filters, listbox, and detail panel with Upcoming seeded appointment)

**Narration cue:** “There's no AI anywhere in this second path: it's the identical business method, called directly by a button click. Whatever rule lives inside APT\_Tool\_CancelAppointment applies here automatically, too.”

Why it lands: demonstrates that the AI is a second front door onto the same business logic, not a parallel system that could quietly drift out of sync with what staff can do by hand.

## Beat 6: Out of scope: what the assistant refuses

Type / say:

*"Can you write me a Python script that scrapes a competitor's website?"*

Fires: nothing

(no tool\_calls in the completion)  
↳ Tool Activity panel stays empty; finish\_reason is "stop" on the first pass

**Narration cue:** "Nothing fired here. The model didn't reach for a tool and then get blocked; it never proposed one, because the system prompt puts everything outside appointment booking off the table. And notice the reply still ends by pointing back at something the clinic can actually do."

Why it lands: the web chat is public and unauthenticated, so scope is the first thing anyone will test. This beat shows the guardrail is the same single mechanism as everything else in the demo (one string, re-injected every turn) rather than a separate moderation layer bolted on in 4D, and that a refused turn costs one short completion with no database access at all.

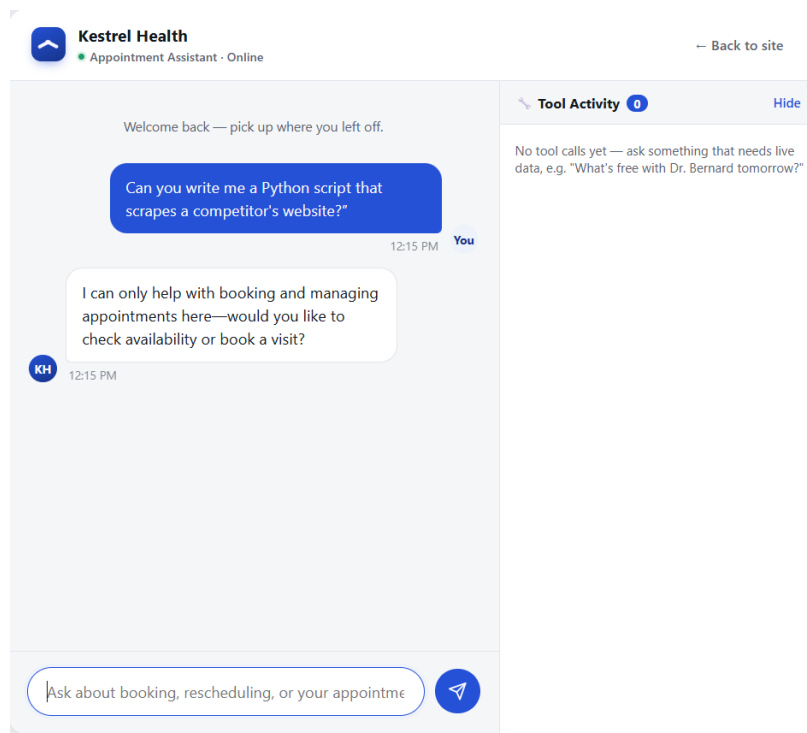


Figure 9: Can you write me a Python script that scrapes a competitor's website?

## Best Practices & Security Considerations

### 6.1 Performance and Scalability

- There's no streaming: a visitor's browser waits for the complete request/response, including any tool-calling rounds, before a reply appears. A client-side typing indicator

covers the wait, an acceptable trade-off for a chat interface, given GPT-5's typical response time of a few seconds per round.

- Each tool call is a full extra request/response cycle to OpenAI. Multi-tool chaining (e.g. findClient → createAppointment) means two or three round trips before the final reply, and the loop is deliberately capped at 6 rounds so a confused model can't spin indefinitely on one HTTP request.
- The ORDA queries behind the tools are indexed on the fields they filter on (staffID, date, status, confirmationCode), keeping tool execution itself fast regardless of model latency; note that client matching (APT\_FindClientEntity) is deliberately done in plain 4D code rather than ORDA's .query() string DSL, since that DSL was found not to support inline field-concatenation comparisons reliably.
- As noted in How the Model Picks a Tool, twelve tools is still comfortably within the range where letting the model reason over the full tool list works well; scaling further toward the twenty-tool mark would call for a semantic pre-filter (embedding-based retrieval) before the completion call, to keep prompt size and selection accuracy under control.

## 6.2 Security

- The OpenAI API key is never embedded in code or in webchat.html; it lives in the gitignored Resources/AIProvider.json and is read server-side via APT\_GetOpenAIKey; only Resources/AIProvider.example.json (no real key) is committed.
- UUIDs are treated as internal identifiers only; every tool that surfaces information back to the client already returns human-readable fields (staff name, confirmation code) instead of raw keys.
- The public /rest/\$catalog/chat endpoint is unauthenticated in this demo; anyone can call it. The system-prompt scope guard and the max\_tokens/round caps limit cost exposure, but they are not an access-control mechanism; a production deployment of a public-facing tool-calling endpoint should add real request authentication/rate-limiting in front of it.
- createAppointment re-checks the requested slot against existing Appointment rows before inserting, guarding against a stale slot suggestion (from a few turns earlier, or from a race between two concurrent conversations) turning into a double-booking.
- Refresh the system message every turn rather than trusting whatever was stored when a conversation began. Otherwise, a resumed conversation (particularly likely on the public site, whose conversationID persists in the browser's localStorage across days) keeps stale rules and a stale "today's date" baked in from whenever it started.

## 6.3 Tool Design Best Practices

- One tool, one responsibility: demoAPT keeps cancelAppointment and rescheduleAppointment separate rather than one overloaded updateAppointment, so the model's choice between them is unambiguous.
- Write descriptions for the model, not for a developer reading the code: description is the only signal the model uses to pick a tool, so it should state exactly when to call it

and any preconditions (see `createAppointment`'s “`findClient` must be called first”, or `getWeekAvailability`'s explicit “use this to proactively suggest alternatives”).

- Keep parameter schemas strict: mark fields required only when the method truly needs them, and document the expected format inline (“YYYY-MM-DD”, “HH:MM”).
- Return small, structured JSON objects from each tool, not free text, and design the reply format around how it will be rendered: the `<slots>` marker convention exists specifically so the model's output can be split into a short sentence plus machine-readable options that the client renders as buttons, rather than the model prose-describing a list the UI then has to re-parse.
- Cap tool-calling loops explicitly, even server-side ones: `APT_WebChat_RunTurn`'s 6-round limit exists specifically so a model that gets stuck calling tools in circles fails with a clear “took too many steps” message instead of hanging the HTTP request indefinitely.

## 6.4 Troubleshooting Common Issues

### A booking lands on the wrong day of the week

4D's `Day of` command returns the day of the month (1 to 31), not the weekday. `checkAvailability` and `APT_ComputeFreeSlots` need the weekday to match against `Availability.dayOfWeek`, so they use `Day number` instead (1 = Sunday ... 7 = Saturday) and convert it to this project's Monday-first convention. Reusing `Day of` by mistake in a new tool will silently compute availability for the wrong day rather than raising an error.

### A reply or database field contains the literal text "null"

`String(Null)` returns the three-character text “null”, not an empty string. A tool handler that calls `String()` on an optional field (reason, email, phone) without first checking it against `Null` risks writing the word “null” into the database or echoing it back to the client. `APT_TextOrEmpty` exists specifically to guard against this before a value reaches ORDA or the model.

## Conclusion

`demoAPT` shows that tool calling removes an entire category of hand-written logic from a conversational 4D application: no intent classifier, no keyword matching, no manual routing of “what does the user want.” The developer's job shifts to describing capabilities clearly (tool schemas), writing the ORDA logic behind each one, and letting the model handle the natural-language side end to end. That includes multi-turn context, multi-step chains (resolving a client before booking, for example), and proactively offering alternatives when a slot is unavailable.

Crucially, `demoAPT` also shows that the same tool definitions, dispatcher, and business methods aren't reserved for the AI conversation alone: the native Admin form calls the identical `APT_Tool_*` methods directly, with no model involved. Staff and AI therefore can never fall out of sync on what a cancellation, reschedule, or lookup does. The same architecture (tool definitions, a dispatcher, and ORDA-backed business methods reachable from more than one front door) could be reusable in any 4D application that needs to expose its data and actions through natural language as well as through forms and menus.